COMPSCI 713: AI Fundamentals

Lecture 11: NEAT

NeuroEvolution of Augmenting Topologies

Instructor: Xinyu Zhang
Adapted from lecture material by Prof. Jim Warren

School of Computer Science, University of Auckland

S1 2026

Overview

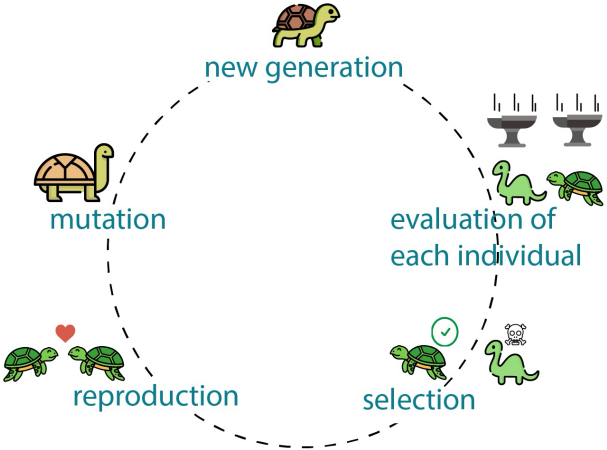
- We'll talk about Genetic Algorithms (GA) and focus on NEAT as an example.

Genetic Algorithms

GAs are a class of algorithms that search for a solution in ways inspired by Darwin's theory of evolution and the principles of genetics.

Key aspects of a GA:

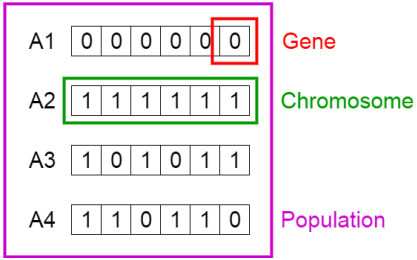
- Initialization
- Fitness / Evaluation
- Selection
- Crossover / Reproduction
- Mutation



From: https://www.generativedesign.org/02-deeper-dive/02-04_genetic-algorithms/02-04-01_what-is-a-genetic-algorithm

Population, genes and chromosomes

- A population consists of N individuals (a chromosome) each with a set of genes.
- How the genes code for phenotype is problem-specific.
  - E.g. might be feature weights in a regression, or represent presence or absence of specific behaviours in a classification.
- Gene values assigned randomly in the initial population.



From: https://towardsdatascience.com/genetic-algorithm-6aefd897f1ac/

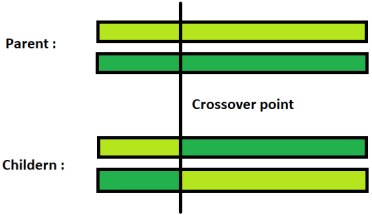
Fitness function, evaluation and selection

- A Fitness Function evaluates how close a solution is to ideal for a given problem.
- In a GA, the fitness function assigns a fitness to each individual.
  - In a regression problem, it might be squared error.
  - In a classification problem it might be accuracy or F1 score.
  - For simulation and gaming, it might be how many turns the individual survived in the environment ("reinforcement learning").
- The fittest individuals are allowed/**selected** to **pass** on their genes to the **next generation**.

Crossover

- The genes of the fittest individuals from evaluation are mixed to create the individuals of the next generation.
- Many crossover methods are possible.
- If preserving a set of genes in order matters for the fitness of the phenotype, we might choose a crossover point.
- We could also allow a small number of very fit individuals to pass through to the next generation directly (called Elitism).

Q: Do all individuals undergo crossover?
A: No.

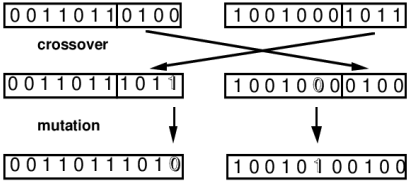


Single-point Crossover

Uniform Crossover

Mutation

- In addition to using crossover to generate unique new individuals in the population, we can apply **mutation** operators.
  - randomly flip a gene (if they are bits)
  - perturb or randomly-reassign it if they are weights.
- Mutation rate – the probability of mutation for a gene (or an individual).
  - Generally will be quite low, e.g. 0.01 or 0.001.
- Both crossover and mutation are genetic operators (let's remember that emulating aspects of Darwinian evolution is just an inspiring metaphor).
  - How much to rely on one strategy or the other is problem-dependent.
- Overall, there are a LOT of parameters to explore when applying GAs (possible genetic operators, mutation rates, population size, length and nature of fitness tests).



GA Applications

- Configuration and scheduling
- Financial portfolio optimization
- Vehicle routing
- Protein folding
- Basically, it's a way of searching a space for a solution.
  - Useful for **vast search spaces** where traditional (breadth first, depth first, etc.) searching is impractical.
  - Works because some intuition for the domain is embedded in the fitness function and approach to crossover and mutation.

The NEAT algorithm: A GA for evolving neural networks

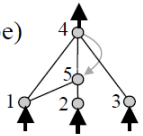
- The chromosome ("genome") of each individual codes for its nodes and edges.
- Starts with:
  - All inputs connected to the output (no hidden nodes – node 5 below would've been added by mutation).
  - Random weights.

Genome (Genotype)

Node Genes	Node 1 Sensor	Node 2 Sensor	Node 3 Sensor	Node 4 Output	Node 5 Hidden

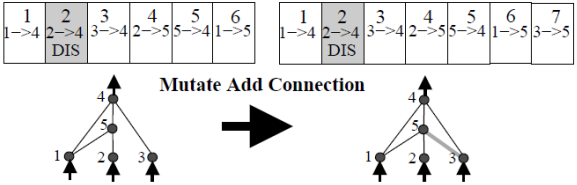
Connect. Genes	In 1 Out 4 Weight 0.7 Enabled Innov 1	In 2 Out 4 Weight-0.5 **DISABLED** Innov 2	In 3 Out 4 Weight 0.5 Enabled Innov 3	In 2 Out 5 Weight 0.2 Enabled Innov 4	In 5 Out 4 Weight 0.4 Enabled Innov 5	In 1 Out 5 Weight 0.6 Enabled Innov 6	In 4 Out 5 Weight 0.6 Enabled Innov 11

Network (Phenotype)



Structural Mutation in NEAT: Add connection

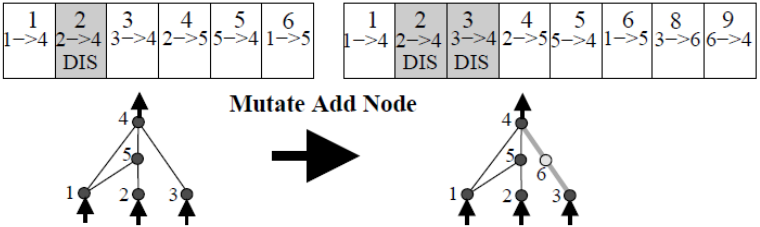
- A new connection gene is added at the end of the genome.
- It can "loop back" (connect to itself or to another hidden node closer to the output), creating a **recurrent** connection. This introduces a feedback loop, so the network's current output can depend on its **previous** internal state (i.e., it can remember past information).
- It gets the next available **Innovation Number** (will be used in Crossover) and a **random weight**.



Connection weight can also mutate (non-structural mutation) as in earlier neuro-evolutionary system.

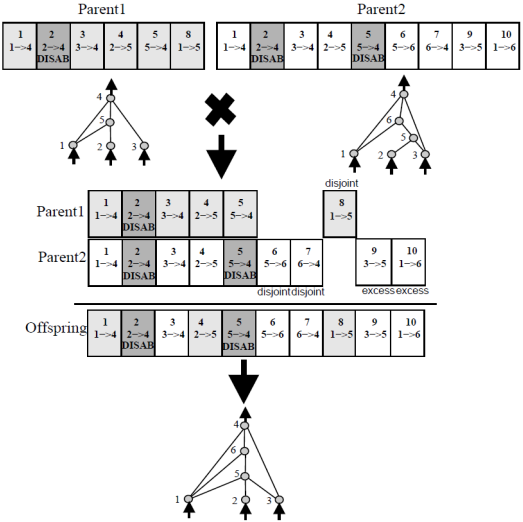
Structural Mutation: Add Node

- A connection gene is disabled, and two new connection genes are added with a new node in between.
- The new connection into the new node gets a weight of 1, and the connection leading out gets the weight of the old connection (thus, initially having the same performance as the pre-mutation individual).



Crossover

- Genes of the two parents are lined up by innovation number.
- Non-matching genes ("**disjoint**" if in the middle of the chromosome, or "**excess**" at the end) are inherited from the fittest parent.
- Or randomly if parents are of equal fitness [as in figure].
- Disabled gene status **inherited** from fittest parent with small random chance of being re-enabled.



Speciation

NEAT includes the concept of species from Darwinian evolution in order to **protect innovation**.

- When a new structure appears, it may initially perform worse than well-established networks.
- Without speciation, it would be eliminated immediately. By grouping similar individuals into species, new structures can compete within their own group and have time to improve.

- A distance between any individuals is computed as:

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \overline{W}$$

- Where E and D are number of excess and disjoint genes, $\overline{W}$ is average weight differences of matching genes; c_1, c_2 and c_3 are configurable parameters.
- Within a threshold δ_t two individuals are compatible to be in the same species.

Assigning species and breeding quotas

- Individuals in a new generation are matched into a species based on **distance** from randomly-chosen example individuals from the previous generation's species.
  - Or form a new species if they aren't compatible with the earlier ones.
- **Fitness is adjusted by dividing by the number of individuals in the species.**
  - adjust fitness $= \frac{\text{individual fitness}}{\text{species size}}$.
  - E.g., Species A: fitness: 10, 8, 6, 12, 9; adjusted fitness: 2, 1.6, 1.2, 2.4, 1.8
  - Species B: fitness: 10, 8; adjusted fitness: 5, 4
- Each species is assigned a quota of individuals in the next generation proportional **to the sum of its adjusted fitness**.
  - Thus, **fitter** species can get **more** individuals, but there's also a pressure against having too many individuals of the same species in the population.
  - Very **unfit** species can **die out**, but a species with few individuals is helped to hang on by having smaller denominator for adjusted fitness.
  - Species A: $2 + 1.6 + 1.2 + 2.4 + 1.8 = 9$; Species B: $5 + 4 = 9$. Quota: 1:1. They receive equal reproduction quotas.

Evaluation task

- The evaluation task is the **environment** used to test each neural network. In this case, each network controls a cart balancing two poles, and its fitness is determined by **how long** it can keep the system stable.
- Double pole balance
  - Cart moving back and forth on a limited length of track, balancing two poles of different lengths.
  - Harder version: without having **angular velocities** of the poles or **velocity** of the cart as inputs.
  - This requires algorithm to add a recurrent node to **detect change in angle between time steps**.

$$\text{angular velocity} \approx \frac{\Delta\theta}{\Delta t}$$

Fitness is number of time steps survived without pole angles exceeding a threshold while keeping cart on bounds of track

Additional fitness criterion penalized oscillation (needed to minimize sum of velocity, angular velocity and angle)

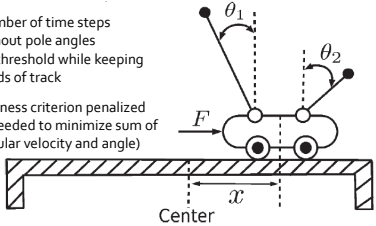


Illustration from Kasshun et al. (2008)

Pipeline

Initialize population of simple neural networks

$\downarrow$

Each network controls the cart in the simulation (outputs force)

$\downarrow$

Run simulation and measure fitness (time survived)

$\downarrow$

Select high-fitness networks (selection)

$\downarrow$

Apply crossover and mutation

$\downarrow$

Add nodes/connections (increase complexity)

$\downarrow$

Form next generation (networks become more complex over time)

$\downarrow$

Repeat over many generations

$\downarrow$

Evolve a network that balances the poles

Evaluation results and ablation analysis

- NEAT mastered the task more quickly than previous neuro-evolutionary methods (which were the best solution methods at the time).
- Ablation experiments showed that the innovative components of NEAT each contributed to performance.
  - In ablation, you leave something out and compare it to the full version.
  - Stanley & Miikkulainen tried using 1) a fixed fully-connected network, 2) starting with a larger-than-minimal network, 3) disabling speciation and 4) disabling crossover (just use mutation).
  - All ablated versions were slower to learn or even failed to master the task.

NEAT applications

- NEAT is good for the things GAs are good for – e.g. searching for good configurations, schedules, traversal paths, etc.
- But NEAT has particular currency for game play strategies.
  - Explainer video for "flappy bird" NEAT solution: https://www.youtube.com/watch?v=OGHA-elMrxI
  - Fun video for NEAT playing Monopoly: https://www.youtube.com/watch?v=dkvFcYBznPI
- Extra interest because of the small neural networks it generates.
  - Maybe this is helpful for explainability.

Conclusion

- Genetic Algorithm provide a mechanism to search out solutions based on iterative refinement.
  - Driven by a fitness function and crossover and mutation of solutions into successive generations.
- NEAT is a specific GA approach that creates neural networks with small numbers of nodes to solve problems.
  - It's shown particular strength for control and game play.
- We've seen a range of Soft Computing strategies to create AIs based on uncertain or approximate solutions.
  - Some are more based in representing knowledge from experts, e.g., Fuzzy Logic.
  - Some are more based in machine learning from labelled data or via reinforcement learning, e.g., Naïve Bayes, NEAT.

References

- Kassahun, Y., Metzen, J. & Kirchner, F. (2008) "Accelerating neuroevolutionary methods using a Kalman filter." Proc GECCO '08, July.
- Stanley, K. & Miikkulainen, R. (2002) "Evolving neural networks through augmenting topologies," Evolutionary Computation 10(2): 99–127.
- Wittkamp, M., Barone, L., & Hingston, P. (2008) "Using NEAT for continuous adaptation and teamwork formation in Pacman." Proc IEEE Symposium on Computational Intelligence and Games (CIG'08).

Quick Quiz: Check Your Understanding

- **Q1.** What is the main purpose of **mutation** in a Genetic Algorithm?

  A. Preserve the best individuals
  B. Introduce new variations into the population
  C. Select the fittest individuals

Quick Quiz: Check Your Understanding

- **Q2.** What is the advantage of adding a **recurrent connection** in NEAT?
  A. It reduces the number of nodes
  B. It allows the network to remember past information
  C. It speeds up crossover

Quick Quiz: Check Your Understanding

- **Q3.** What is the purpose of **speciation** in NEAT?
  A. To increase mutation rate
  B. To protect new structures from being eliminated too early
  C. To reduce population size

Thank you!
Q&A